

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#) ▶ [Meus cursos](#) ▶ [PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#) ▶ [Tópico 1](#) ▶

[Análise teórica de eficiência](#)

Análise teórica de eficiência

Complexidade de algoritmos

- Analisar algoritmos envolve decompô-los em partes menores e estudar sua eficiência em termos de tempo e espaço.
- A complexidade de tempo é uma função que fornece o tempo de execução para cada tamanho de entrada, enquanto a complexidade de espaço é uma função que fornece o consumo de memória para cada tamanho de entrada.
- O tamanho da entrada pode ser medido determinando o número total de bits usados em sua representação binária, ou usando o número de elementos em uma coleção de elementos com um número máximo de bits por elemento.
- Na análise teórica, o tempo é estimado contando o número total de instruções executadas, ou contando apenas o número de execuções das operações básicas (as mais importantes na análise).

EXEMPLO: ordenação por seleção

```
Algoritmo selectionSort(A)
Pré-condições: A é um array de inteiros
Pós-condições: elementos de A estão em ordem não decrescente
1   for i from 0 to length(A)-1      n
2       indexMin = i                 n
3       for j from i+1 to length(A)-1  n(n-1)/2
4           if A[j] < A[indexMin] then  n(n-1)/2
5               indexMin = j           n(n-1)/2
6       troca A[i] com A[indexMin]    n
```

- Linhas 2, 4, 5 e 6 executam em tempo constante em cada iteração.
- Seja n o tamanho da entrada (número de elementos em A).
- O 'for' da linha 1 é executado n vezes.
- O 'for' da linha 3 é executado (n-1) - ((i+1) - 1) = n-i-1 vezes, para cada valor de i.
- Soma de n-i-1 para i indo de 0 até n-1. Isto vale n(n-1)/2.
- Total de instruções: 3n + 3n(n-1)/2 = 1.5n^2 + 1.5n.
- A taxa de crescimento da complexidade de um algoritmo determina o quão rápido ele cresce com o tamanho da entrada.
- A tabela mostra o tempo de execução para cada tamanho de entrada em função das principais taxas de crescimento da complexidade, assumindo que um computador executa 1 bilhão de instruções por segundo.

Tempo(segundos) = $\frac{\text{Complexidade (instruções)}}{\text{Velocidade (instruções/segundo)}}$

n	log n	n	n log n	n^2	n^3	2^n	n!
10	3ns	10ns	33ns	100ns	1us	1u s	4ms
10^2	7ns	100ns	7us	10us	1ms	10^13a	10^141a
10^3	10ns	1us	10us	1ms	1s		
10^4	13ns	10us	130us	100ms	17m		
10^5	17ns	100us	2ms	10s	12d		
10^6	20ns	1ms	20ms	17min	32a		

EXEMPLO: (anterior) Algoritmo tem tempo quadrádico (n^2).

- A análise de pior caso considera a pior entrada possível de tamanho n para definir um limite superior no tempo de execução.
- A análise de melhor caso considera a entrada que produz o menor tempo de execução entre todas as entradas de tamanho n e pode revelar tipos de entrada úteis com bom desempenho.
- A análise de caso médio leva em consideração entradas "aleatórias" e geralmente assume que todas as entradas possíveis têm a mesma chance de ocorrer.

=====

EXERCÍCIO: Considere o código python abaixo que multiplica uma matriz (m linhas e m colunas) por um vetor (m elementos).

- Determine a complexidade (total de instruções executadas) em função de m.
- Qual o tamanho da entrada n? Determine o total de instruções em função de n.
- Qual a taxa de crescimento da complexidade em função de m e de n?

```
def multiply_matrix_vector(matrix, vector):
    result = []
    for i in range(len(matrix)):
        sum = 0
        for j in range(len(vector)):
            sum += matrix[i][j] * vector[j]
        result.append(sum)
    return result

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
vector = [1, 2, 3]

result = multiply_matrix_vector(matrix, vector)
print(result)
```

Última atualização: domingo, 21 mai 2023, 09:05

[◀ Registro de presença](#)

Seguir para...

Teste ▶

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 Baixar o aplicativo móvel.